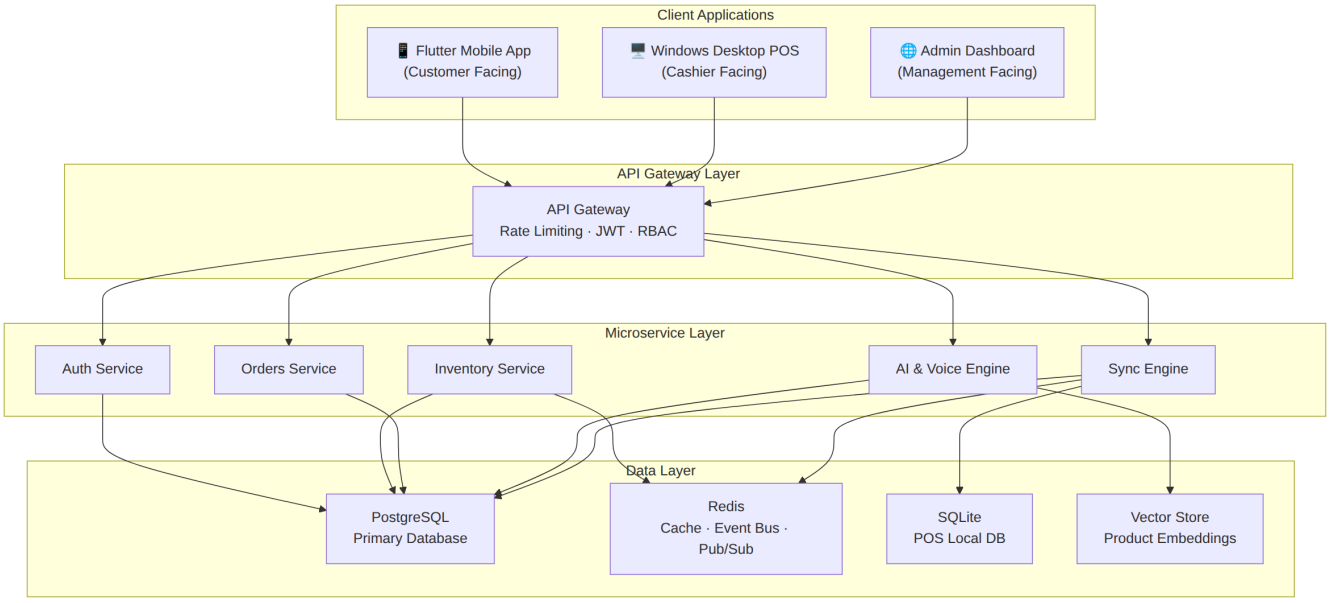
🏗️ Cyborg Supermarket — System Architecture & Implementation Blueprint

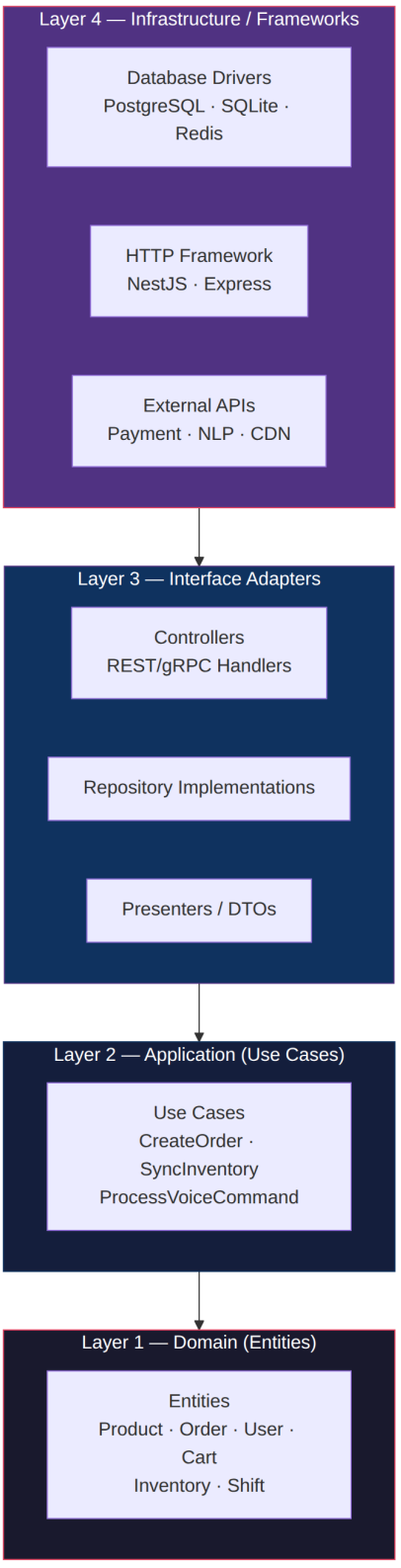
1. System Overview

Cyborg is an **omni-channel Egyptian supermarket ecosystem** consisting of three interconnected platforms, a shared backend, and an AI intelligence layer — all synchronized in real-time.



2. Design Pattern: Clean Architecture (Onion)

Every component in the system follows **Clean Architecture** — ensuring that business logic is completely decoupled from frameworks, databases, and UI.



Dependency Rule: Dependencies point **inward only**. Domain entities NEVER depend on outer layers. Repositories are defined as interfaces in the domain and implemented in infrastructure.

Additional Patterns Used

Pattern	Where Applied	Purpose
Repository	All data access	Abstracts database from business logic

Pattern	Where Applied	Purpose
BLoC / Cubit	Flutter Mobile App	State management with unidirectional data flow
CQRS	Orders & Inventory	Separates read/write models for performance
Event Sourcing	Inventory Sync	Tracks every stock change as immutable events
Observer / Pub-Sub	Redis Event Bus	Real-time sync between POS ↔ Cloud
Strategy	AI Engine	Swappable NLP/embedding providers
Factory	Order Processing	Creates different order types (POS, Online, Family)
Adapter	Hardware Integration	Barcode scanner, thermal printer abstraction
Singleton	DB Connections, Redis	Single shared instance per process
DTO	API Boundaries	Data Transfer Objects for type-safe API contracts

3. File Organization — Full Monorepo Structure

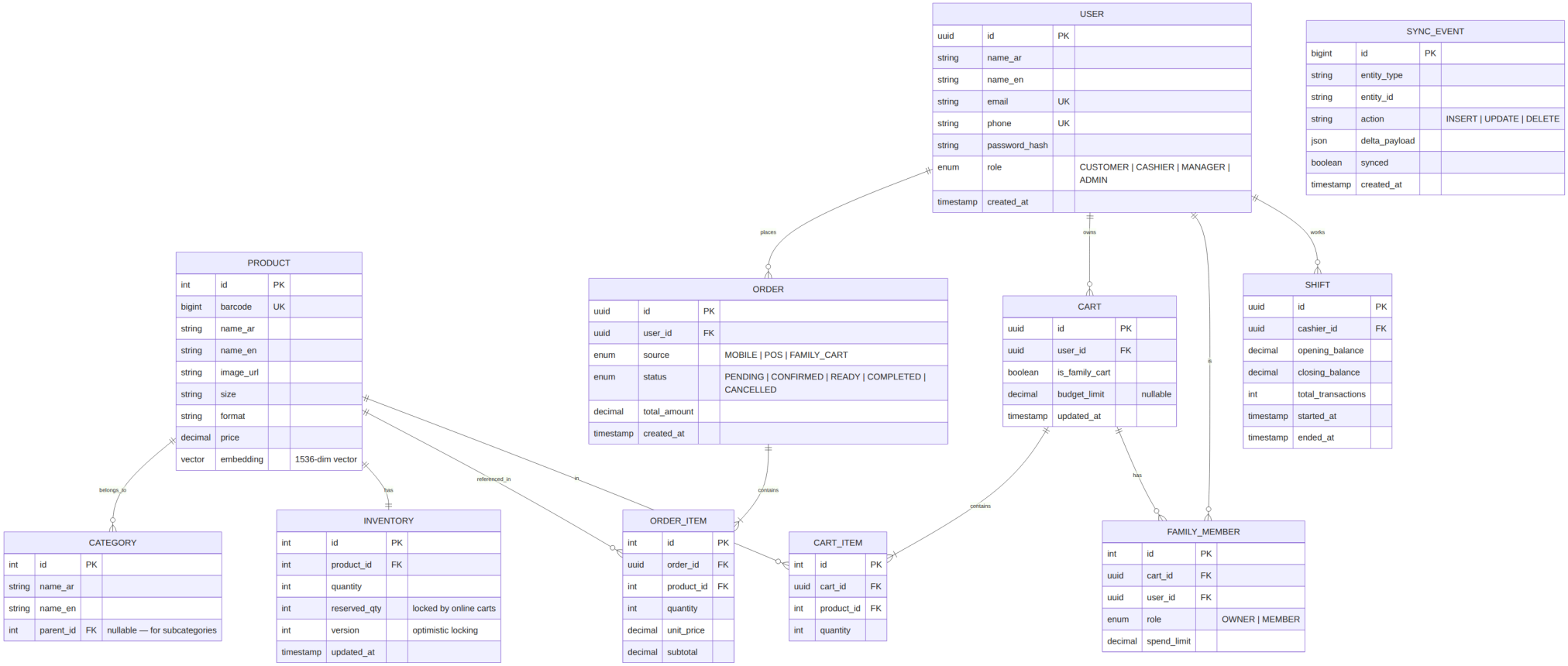
```
cyborg-supermarket/
│
├── 📁 .github/
│   └── workflows/
│       ├── ci-backend.yml            # Backend CI pipeline
│       ├── ci-mobile.yml             # Flutter CI pipeline
│       ├── ci-pos.yml                # POS CI pipeline
│       └── cd-deploy.yml             # Production deployment
│
├── 📁 docs/
│   ├── architecture.md               # System architecture docs
│   ├── api-contracts.md              # REST/gRPC API specs
│   ├── database-erd.md               # Entity Relationship Diagrams
│   └── runbooks/                     # Operational runbooks
│
├── 📁 data/
│   └── super_market.xlsx             # Seed data (1000 products)
│
├═══════════════════════════════════════════════════════════
│                        BACKEND
├═══════════════════════════════════════════════════════════
├── 📁 backend/
│   ├── src/
│   │   ├── main.ts                   # Application entry point
│   │   ├── app.module.ts             # Root NestJS module
│   │   │
│   │   ├── 📁 config/
│   │   │   ├── env.config.ts         # Environment variables
│   │   │   ├── database.config.ts    # DB connection config
│   │   │   ├── redis.config.ts       # Redis connection config
│   │   │   └── jwt.config.ts         # JWT secret & expiry
│   │   │
│   │   ├── 📁 common/
│   │   │   ├── decorators/           # Custom decorators (@Roles, @Public)
│   │   │   ├── filters/              # Exception filters
│   │   │   ├── guards/               # Auth & RBAC guards
│   │   │   ├── interceptors/         # Logging, transform interceptors
│   │   │   ├── pipes/                # Validation pipes
│   │   │   └── dto/                  # Shared DTOs
│   │   │
│   │   ├── 📁 database/
│   │   │   ├── migrations/           # Sequential DB migrations
│   │   │   ├── seeders/              # Data seeding scripts
│   │   │   │   └── product-seeder.ts  # Import from super_market.xlsx
│   │   │   ├── entities/             # TypeORM/Prisma entity models
│   │   │   │   ├── user.entity.ts
│   │   │   │   ├── product.entity.ts
│   │   │   │   ├── category.entity.ts
│   │   │   │   ├── order.entity.ts
│   │   │   │   ├── order-item.entity.ts
│   │   │   │   ├── inventory.entity.ts
│   │   │   │   ├── cart.entity.ts
│   │   │   │   ├── family-cart.entity.ts
```

```
│   │   │   │   └── shift.entity.ts
│   │   │   └── repositories/        # Repository implementations
│   │   │
│   │   ├── 📁 modules/
│   │   │   ├── 📁 auth/
│   │   │   │   ├── auth.module.ts
│   │   │   │   ├── auth.controller.ts
│   │   │   │   ├── auth.service.ts
│   │   │   │   ├── strategies/       # JWT, Local, OAuth strategies
│   │   │   │   ├── dto/              # LoginDto, RegisterDto
│   │   │   │   └── tests/
│   │   │   │
│   │   │   ├── 📁 users/
│   │   │   │   ├── users.module.ts
│   │   │   │   ├── users.controller.ts
│   │   │   │   ├── users.service.ts
│   │   │   │   ├── dto/
│   │   │   │   └── tests/
│   │   │   │
│   │   │   ├── 📁 products/
│   │   │   │   ├── products.module.ts
│   │   │   │   ├── products.controller.ts
│   │   │   │   ├── products.service.ts
│   │   │   │   ├── dto/
│   │   │   │   └── tests/
│   │   │   │
│   │   │   ├── 📁 inventory/
│   │   │   │   ├── inventory.module.ts
│   │   │   │   ├── inventory.controller.ts
│   │   │   │   ├── inventory.service.ts
│   │   │   │   ├── inventory.gateway.ts  # WebSocket gateway
│   │   │   │   ├── dto/
│   │   │   │   └── tests/
│   │   │   │
│   │   │   ├── 📁 orders/
│   │   │   │   ├── orders.module.ts
│   │   │   │   ├── orders.controller.ts
│   │   │   │   ├── orders.service.ts
│   │   │   │   ├── factories/        # OrderFactory (POS, Online, Family)
│   │   │   │   ├── dto/
│   │   │   │   └── tests/
│   │   │   │
│   │   │   ├── 📁 cart/
│   │   │   │   ├── cart.module.ts
│   │   │   │   ├── cart.controller.ts
│   │   │   │   ├── cart.service.ts
│   │   │   │   ├── family-cart.service.ts
│   │   │   │   ├── dto/
│   │   │   │   └── tests/
│   │   │   │
│   │   │   ├── 📁 ai-engine/
│   │   │   │   ├── ai-engine.module.ts
│   │   │   │   ├── nlp/
│   │   │   │   │   ├── arabic-nlp.service.ts     # Egyptian Arabic parser
│   │   │   │   │   └── intent-resolver.ts        # Voice command intents
│   │   │   │   ├── embeddings/
│   │   │   │   │   ├── embedding.service.ts      # Vector embeddings
│   │   │   │   │   └── substitution.service.ts   # Product substitution
│   │   │   │   ├── pantry/
│   │   │   │   │   └── smart-pantry.service.ts   # Pantry tracking
│   │   │   │   └── tests/
│   │   │   │
│   │   │   └── 📁 sync/
│   │   │       ├── sync.module.ts
│   │   │       ├── sync.gateway.ts     # WebSocket gateway for POS
│   │   │       ├── delta-sync.service.ts
│   │   │       ├── event-bus.service.ts  # Redis pub/sub
│   │   │       └── tests/
│   │   │
│   │   └── 📁 shared/
│   │       ├── interfaces/           # Domain interfaces
│   │       ├── enums/                # Shared enums (Roles, OrderStatus)
│   │       ├── constants/            # App constants
│   │       └── utils/                # Utility functions
│   │
│   ├── test/                         # E2E tests
│   ├── Dockerfile
│   ├── package.json
│   ├── tsconfig.json
│   └── nest-cli.json
│
```

```
┌──────────────────────────────────────────────────────────────┐
│                    FLUTTER MOBILE APP                        │
└──────────────────────────────────────────────────────────────┘
├── 📁 mobile/
│   ├── lib/
│   │   ├── main.dart                # App entry point
│   │   │
│   │   ├── 📁 app/
│   │   │   ├── app.dart             # MaterialApp config
│   │   │   ├── themes/
│   │   │   │   ├── app_theme.dart   # Light/Dark theme definitions
│   │   │   │   ├── colors.dart      # Color palette
│   │   │   │   └── typography.dart  # Text styles
│   │   │   ├── routes/
│   │   │   │   ├── app_router.dart  # Route definitions
│   │   │   │   └── route_guards.dart # Auth guards
│   │   │   └── di/
│   │   │       └── injection.dart   # Dependency injection (get_it)
│   │   │
│   │   ├── 📁 core/
│   │   │   ├── network/
│   │   │   │   ├── dio_client.dart  # HTTP client setup
│   │   │   │   ├── api_endpoints.dart
│   │   │   │   ├── interceptors/    # Auth, error, logging interceptors
│   │   │   │   └── websocket_client.dart
│   │   │   ├── storage/
│   │   │   │   └── local_storage.dart  # SharedPrefs / Hive
│   │   │   ├── errors/
│   │   │   │   ├── failures.dart
│   │   │   │   └── exceptions.dart
│   │   │   └── utils/
│   │   │       ├── validators.dart
│   │   │       └── formatters.dart
│   │   │
│   │   ├── 📁 features/
│   │   │   ├── 📁 auth/
│   │   │   │   ├── data/
│   │   │   │   │   ├── models/      # UserModel (JSON serializable)
│   │   │   │   │   ├── datasources/ # Remote & local data sources
│   │   │   │   │   └── repositories/ # AuthRepositoryImpl
│   │   │   │   ├── domain/
│   │   │   │   │   ├── entities/    # User entity
│   │   │   │   │   ├── repositories/ # AuthRepository (abstract)
│   │   │   │   │   └── usecases/    # Login, Register, Logout
│   │   │   │   └── presentation/
│   │   │   │       ├── bloc/        # AuthBloc, AuthState, AuthEvent
│   │   │   │       ├── pages/       # LoginPage, RegisterPage
│   │   │   │       └── widgets/     # LoginForm, SocialButtons
│   │   │   │
│   │   │   ├── 📁 product_catalog/
│   │   │   │   ├── data/
│   │   │   │   ├── domain/
│   │   │   │   └── presentation/
│   │   │   │       ├── bloc/        # ProductBloc
│   │   │   │       ├── pages/       # CatalogPage, ProductDetailPage
│   │   │   │       └── widgets/     # ProductCard, CategoryFilter, SearchBar
│   │   │   │
│   │   │   ├── 📁 cart/
│   │   │   │   ├── data/
│   │   │   │   ├── domain/
│   │   │   │   └── presentation/
│   │   │   │       ├── bloc/        # CartBloc
│   │   │   │       ├── pages/       # CartPage, CheckoutPage
│   │   │   │       └── widgets/     # CartItem, PriceSummary
│   │   │   │
│   │   │   ├── 📁 family_cart/
│   │   │   │   ├── data/
│   │   │   │   ├── domain/
│   │   │   │   └── presentation/
│   │   │   │       ├── bloc/        # FamilyCartBloc
│   │   │   │       ├── pages/       # FamilyCartPage, BudgetPage
│   │   │   │       └── widgets/     # MemberList, BudgetBar
│   │   │   │
│   │   │   ├── 📁 ai_assistant/
│   │   │   │   ├── data/
│   │   │   │   ├── domain/
│   │   │   │   └── presentation/
│   │   │   │       ├── bloc/        # VoiceBloc
│   │   │   │       ├── pages/       # AssistantPage
│   │   │   │       └── widgets/     # VoiceRecordButton, SuggestionChips
│   │   │   │
```

```
│   │   │   └── 📁 smart_pantry/
│   │   │       ├── data/
│   │   │       ├── domain/
│   │   │       └── presentation/
│   │   │
│   │   └── 📁 shared/
│   │       ├── widgets/              # Reusable widgets
│   │       ├── extensions/           # Dart extensions
│   │       └── constants/
│   │
│   ├── test/
│   │   └── features/                # Widget & integration tests
│   ├── pubspec.yaml
│   └── analysis_options.yaml
│
├──────────────────────────────────────────────────────────
│                   WINDOWS POS CLIENT
├──────────────────────────────────────────────────────────
├── 📁 pos/
│   ├── lib/
│   │   ├── main.dart
│   │   │
│   │   ├── 📁 core/
│   │   │   ├── database/
│   │   │   │   ├── sqlite_helper.dart      # Local SQLite connection
│   │   │   │   └── pos_migrations.dart     # Local schema migrations
│   │   │   ├── hardware/
│   │   │   │   ├── barcode_scanner.dart    # Scanner adapter
│   │   │   │   ├── thermal_printer.dart    # Receipt printer adapter
│   │   │   │   └── cash_drawer.dart        # Cash drawer controller
│   │   │   └── sync/
│   │   │       ├── sync_engine.dart        # Delta sync (SQLite ↔ Cloud)
│   │   │       └── conflict_resolver.dart  # Merge conflict strategy
│   │   │
│   │   ├── 📁 features/
│   │   │   ├── 📁 checkout/
│   │   │   │   ├── data/
│   │   │   │   ├── domain/
│   │   │   │   └── presentation/
│   │   │   │       ├── bloc/
│   │   │   │       ├── pages/              # ScanPage, PaymentPage
│   │   │   │       └── widgets/            # CartPanel, BarcodeInput
│   │   │   │
│   │   │   ├── 📁 shift_management/
│   │   │   │   ├── data/
│   │   │   │   ├── domain/
│   │   │   │   └── presentation/
│   │   │   │       ├── pages/              # ShiftStartPage, ShiftSummaryPage
│   │   │   │       └── widgets/
│   │   │   │
│   │   │   └── 📁 inventory_lookup/
│   │   │       ├── data/
│   │   │       ├── domain/
│   │   │       └── presentation/
│   │   │
│   │   └── 📁 shared/
│   │
│   ├── test/
│   ├── pubspec.yaml
│   └── windows/                      # Windows runner config
│
├──────────────────────────────────────────────────────────
│                     INFRASTRUCTURE
├──────────────────────────────────────────────────────────
├── docker-compose.yml                # Local dev: Backend + PG + Redis
├── docker-compose.prod.yml           # Production overrides
├── 📁 infra/
│   ├── Dockerfile.backend
│   ├── Dockerfile.pos
│   ├── nginx/
│   │   └── nginx.conf                # Reverse proxy config
│   ├── terraform/                    # Cloud IaC (optional)
│   └── monitoring/
│       ├── prometheus.yml
│       └── grafana-dashboards/
│
├── .env.example
├── .gitignore
└── README.md
```

4. Database Schema (ERD)


USER
uuid id PK
string name_ar
string name_en
string email UK
string phone UK
string password_hash
enum role CUSTOMER | CASHIER | MANAGER | ADMIN
timestamp created_at
SYNC_EVENT
bigint id PK
string entity_type
string entity_id
string action INSERT | UPDATE | DELETE
json delta_payload
boolean synced
timestamp created_at
places
owns
works
is
PRODUCT
int id PK
bigint barcode UK
string name_ar
string name_en
string image_url
string size
string format
decimal price
vector embedding 1536-dim vector
ORDER
uuid id PK
uuid user_id FK
enum source MOBILE | POS | FAMILY_CART
enum status PENDING | CONFIRMED | READY | COMPLETED | CANCELLED
decimal total_amount
timestamp created_at
CART
uuid id PK
uuid user_id FK
boolean is_family_cart
decimal budget_limit nullable
timestamp updated_at
SHIFT
uuid id PK
uuid cashier_id FK
decimal opening_balance
decimal closing_balance
int total_transactions
timestamp started_at
timestamp ended_at
belongs_to
has
referenced_in
in
contains
contains
has
CATEGORY
int id PK
string name_ar
string name_en
int parent_id FK nullable — for subcategories
INVENTORY
int id PK
int product_id FK
int quantity
int reserved_qty locked by online carts
int version optimistic locking
timestamp updated_at
ORDER_ITEM
int id PK
uuid order_id FK
int product_id FK
int quantity
decimal unit_price
decimal subtotal
CART_ITEM
int id PK
uuid cart_id FK
int product_id FK
int quantity
FAMILY_MEMBER
int id PK
uuid cart_id FK
uuid user_id FK
enum role OWNER | MEMBER
decimal spend_limit

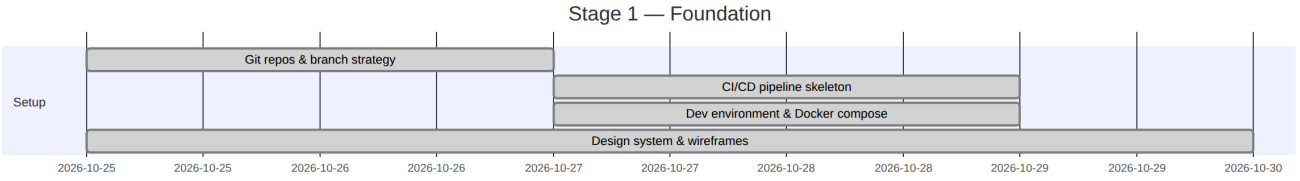
5. Job Roles

Code	Role	Description
BE	Backend Developer	NestJS APIs, database, auth, sync engine
FM	Flutter Mobile Developer	Customer-facing mobile app (iOS & Android)
FP	Flutter POS Developer	Windows desktop POS application
AI	AI/ML Engineer	NLP, voice processing, embeddings, smart features
QA	QA Engineer	Testing, security audit, stress testing
DO	DevOps Engineer	CI/CD, Docker, cloud infra, monitoring
PM	Project Manager	Coordination, milestone tracking, reviews
UI	UI/UX Designer	Design system, wireframes, user flows

6. Staged Workflow with Role-Based Tasks

🟣 Stage 1 — Foundation Setup & Configuration

> **Duration**: ~1 week · **Sprint 0**



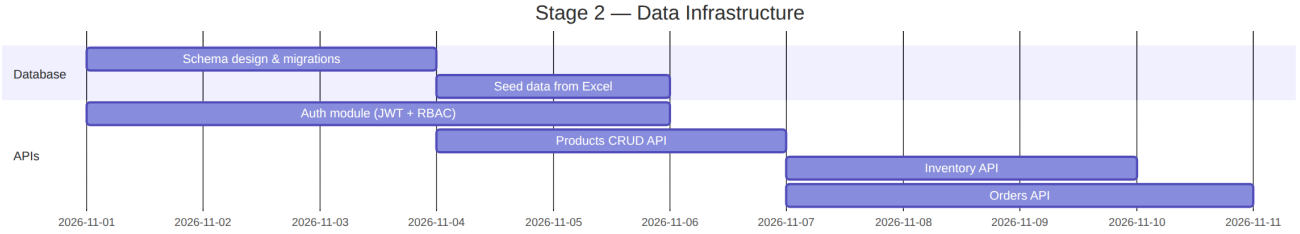
Role	Tasks
PM	• Create Jira/Linear board with all 8 stages as epics • Define sprint cadence (2-week sprints) • Set up team communication channels • Document Definition of Done for each stage
DO	• Initialize Git monorepo with branch protection rules • Set up CI pipelines (lint, test, build) for backend/mobile/POS • Create `docker-compose.yml` with PostgreSQL 16 + Redis 7 • Configure `.env.example` with all env variables • Set up cloud hosting accounts (AWS/GCP)
BE	• Scaffold NestJS project with Clean Architecture folders • Configure `src/config/` — env, database, redis, jwt configs • Set up `src/main.ts` entry with global pipes, filters, interceptors • Install core dependencies: TypeORM/Prisma, Passport, class-validator • Create base repository interface in `src/shared/interfaces/`
FM	• Initialize Flutter project with `lib/app/`, `lib/core/`, `lib/features/` structure • Configure `lib/app/themes/` — color palette, typography, app theme • Set up Dio HTTP client in `lib/core/network/dio_client.dart` • Configure dependency injection (get_it + injectable) • Set up route management (go_router / auto_route)

Role	Tasks
FP	• Initialize Flutter desktop project targeting Windows • Configure `lib/core/database/sqlite_helper.dart` • Research & document barcode scanner SDK integration approach • Set up Windows-specific build configs
UI	• Create design system: color palette, typography scale, spacing • Design wireframes for all major screens (mobile + POS) • Define component library (buttons, cards, inputs, modals) • Create interactive prototypes in Figma for key user flows

> **Stage Gate**: All repos initialized, Docker dev stack runs with `docker-compose up`, both Flutter apps build without errors, and design system is approved.

🔵 Stage 2 — Data Infrastructure & Core API

> **Duration**: ~2 weeks · **Sprint 1**



Role	Tasks
PM	• Review API contract documentation with frontend teams • Coordinate schema review meetings • Track blockers between BE and database design
BE	• Design & implement all entity models in `src/database/entities/` — `user.entity.ts`, `product.entity.ts`, `category.entity.ts`, `inventory.entity.ts`, `order.entity.ts`, `order-item.entity.ts`, `cart.entity.ts`, `shift.entity.ts` • Write database migrations in `src/database/migrations/` • Build `product-seeder.ts` to import 1000 products from `super_market.xlsx` — Map columns: `barcode`, `ArabicProductName`, `EnglishProductName`, `ArabicCategory`, `EnglishCategory`, `ArabicSubCategory`, `EnglishSubCategory`, `brand`, `country`, `size`, `format`, `price`, `image` • Implement Auth module: — `auth.controller.ts`: POST `/auth/login`, POST `/auth/register`, POST `/auth/refresh` — `auth.service.ts`: JWT token generation, password hashing (bcrypt) — JWT strategy + Local strategy in `strategies/` — RBAC guard in `src/common/guards/` • Implement Products module: — CRUD endpoints with pagination, search, category filtering — Arabic + English search support • Implement Inventory module: — Stock level queries, update with optimistic locking • Implement Orders module: — Create order, update status, order history — `OrderFactory` for different order types
FM	• Create data layer models matching API DTOs • Implement `AuthRepository` (abstract + impl) in auth feature • Build API service classes consuming backend endpoints • Set up token storage & auto-refresh interceptor

Role	Tasks
FP	• Design local SQLite schema mirroring cloud products + inventory • Write `pos_migrations.dart` for local DB setup • Implement local product lookup repository
QA	• Write unit tests for all entity validations • Create Postman/Insomnia collection for all API endpoints • Define test data fixtures

Stage Gate: All CRUD APIs return valid responses, JWT auth flow works end-to-end, database seeded with 1000 products (5 categories, 7 subcategories, 143 brands), all unit tests pass.

🟢 Stage 3 — Desktop POS & Cashier System

Duration: ~2 weeks · **Sprint 2**

Role	Tasks
PM	• Coordinate hardware procurement (barcode scanners, thermal printers) • Schedule POS demo with store operations team • Review cashier user flow for UX approval
FP	• Build checkout UI: — `ScanPage` : Full-screen barcode input + scanned item list — `PaymentPage` : Cash/card payment selection, change calculator — `CartPanel` widget: Running total, quantity adjustment, item removal • Implement barcode scanner integration in `core/hardware/barcode_scanner.dart` — Adapter pattern to support multiple scanner brands • Implement thermal printer adapter in `core/hardware/thermal_printer.dart` — Receipt template with store branding, item list, totals • Build shift management feature: — `ShiftStartPage` : Cashier login + opening balance entry — `ShiftSummaryPage` : Closing balance, transaction count, discrepancy report • Implement local cart management using SQLite — Offline-first: all operations work without network — Queue orders for sync when connectivity restores
BE	• Update Orders module to accept POS-originated orders — Add `source: POS` enum value — Add `shift_id` foreign key to orders • Create shift management endpoints: — POST `/shifts/start` , PUT `/shifts/{id}/close` — GET `/shifts/{id}/summary`
UI	• Design POS interface optimized for touch screens — Large tap targets, high contrast, minimal navigation depth • Design receipt layout template
QA	• Test POS offline mode: create orders while disconnected • Test barcode scanning with sample barcodes from product data • Verify shift open/close flow

Stage Gate: Cashier can scan barcodes, build a cart, process payment, print receipt, and close shift — all while offline. Orders queue for sync.

🟡 Stage 4 — Customer Mobile App

Duration: ~2.5 weeks · **Sprint 3**

Role	Tasks
PM	• Review mobile UX with focus group feedback • Prioritize feature backlog for MVP vs. post-launch • Coordinate API contract alignment with BE team
FM	• Build **Auth feature** presentation layer: — `LoginPage` & `RegisterPage` with validation — `AuthBloc` state management (Loading → Authenticated → Error) — Phone number + OTP flow (Egyptian market) • Build **Product Catalog** feature: — `CatalogPage` : Grid/list toggle, category tabs (Fresh Food, Beverages, Frozen Food, Food Cupboard, Fruits & Vegetables) — `ProductDetailPage` : Product image, bilingual name, price, size, add-to-cart — `SearchBar` with Arabic text support — `CategoryFilter` widget — `ProductBloc` with pagination, search, and filtering • Build **Cart** feature: — `CartPage` : Item list, quantity stepper, remove, price summary — `CheckoutPage` : Delivery address, payment method, order confirmation — `CartBloc` managing cart state with add/remove/update • Build **Family Cart** feature: — `FamilyCartPage` : Shared cart view with member contributions — `BudgetPage` : Budget limit setting, spend tracking per member — `FamilyCartBloc` : Budget enforcement, member invite via link/QR — `MemberList` widget with spend limits — `BudgetBar` visual progress indicator
BE	• Implement Cart module: — CRUD endpoints for cart items — Family cart creation, member invite, budget enforcement — Budget validation middleware • Add product search endpoint with Arabic full-text search • Implement pagination with cursor-based or offset pagination
UI	• Finalize mobile UI for all screens • Create micro-animations for cart interactions • Design empty states, loading skeletons, error states
QA	• Test product catalog with all 1000 products • Test cart operations: add, update qty, remove, checkout • Test family cart: multi-user budget enforcement • Test Arabic and English locale switching

> **Stage Gate**: Customer can register, browse the full catalog (bilingual), search products, manage cart, use family cart with budget, and place an order.

🟠 Stage 5 — AI Engine & Smart Features

> **Duration**: ~2 weeks · **Sprint 4**

Role	Tasks
PM	• Define AI feature acceptance criteria • Coordinate with AI engineer on NLP model selection • Plan A/B testing for AI-driven suggestions

Role	Tasks
AI	• Implement Egyptian Arabic NLP service in `src/modules/ai-engine/nlp/` — Speech-to-text integration (Google Cloud Speech / Whisper) — `arabic-nlp.service.ts` : Tokenize, normalize Egyptian dialect — `intent-resolver.ts` : Map voice commands → structured intents — Intent types: `ADD_TO_CART` , `SEARCH_PRODUCT` , `CHECK_PRICE` , `REORDER` • Implement product embeddings in `src/modules/ai-engine/embeddings/` — Generate 1536-dim vectors for all 1000 products — `embedding.service.ts` : Vector similarity search — `substitution.service.ts` : Find similar products when item is out of stock • Implement Smart Pantry in `src/modules/ai-engine/pantry/` — `smart-pantry.service.ts` : Track consumption patterns — Auto-generate replenishment suggestions based on order history
BE	• Create AI engine module wiring & API endpoints: — POST `/ai/voice` — accept audio, return structured intent — GET `/ai/substitutions/{productId}` — return similar products — GET `/ai/pantry/suggestions` — return restock suggestions • Set up vector store (pgvector extension on PostgreSQL) • Create embeddings seeder to process all products
FM	• Build AI Assistant feature: — `AssistantPage` : Voice interaction UI — `VoiceRecordButton` : Press-and-hold microphone with visual feedback — `SuggestionChips` : Display parsed intent as actionable chips — `VoiceBloc` : Record → Upload → Parse → Act pipeline • Integrate substitution suggestions on out-of-stock products • Add Smart Pantry page with suggested shopping list
QA	• Test voice commands with Egyptian Arabic audio samples • Validate substitution accuracy with known product pairs • Test edge cases: unclear speech, mixed Arabic-English

> **Stage Gate**: Voice commands in Egyptian Arabic successfully add items to cart, product substitutions return relevant alternatives, smart pantry generates meaningful suggestions.

🔴 Stage 6 — Cloud-Local Synchronization

> **Duration**: ~2 weeks · **Sprint 5**

Role	Tasks
PM	• Define sync SLA (max latency, conflict resolution policy) • Coordinate stress test scenario design • Review edge case documentation
BE	• Implement Event Bus using Redis Pub/Sub in `sync/event-bus.service.ts` • Build WebSocket gateway in `sync/sync.gateway.ts` — Bi-directional channel: Backend ↔ POS clients — Events: `inventory.updated` , `order.created` , `order.status_changed` • Implement `delta-sync.service.ts` : — Track changes via `SYNC_EVENT` table — Batch sync: POS uploads queued changes on reconnect — Conflict resolution: Last-write-wins with version vectors • Update inventory module: — `inventory.gateway.ts` : Broadcast stock changes via WebSocket — Reserve stock when mobile cart adds items — Deduct stock when POS sale completes — Reconcile reserved vs. actual on timeout

Role	Tasks
FP	• Implement Background Sync Engine in `core/sync/sync_engine.dart` — Delta tracking: log all local changes to sync queue — Reconnection: detect connectivity, upload queue on restore — `conflict_resolver.dart` : Handle version conflicts gracefully • Subscribe to WebSocket for real-time inventory updates • Update local SQLite when cloud pushes changes
FM	• Subscribe to WebSocket for real-time stock updates — Update product availability in catalog without refresh — Notify user if carted item becomes unavailable • Display "X left in stock" real-time badge
QA	• Simulate concurrent operations: — Mobile user and cashier purchasing the same last-in-stock item — POS offline for 30 min, then sync 50+ transactions • Verify no data loss during network interruption • Validate inventory counts match across Cloud DB and all POS SQLite instances

Stage Gate: A sale at the physical POS instantly reflects in the mobile app's stock count. A mobile order reserves inventory visible on the cashier's screen. POS works offline and syncs cleanly on reconnection.

○ Stage 7 — QA Testing & Security Audit

Duration: ~1.5 weeks · **Sprint 6**

Role	Tasks
PM	• Compile test report & bug triage • Prioritize critical/blocker fixes • Schedule security audit review meeting
QA	• **Unit Tests**: — Backend: `src/modules/*/tests/` — all services & controllers — Mobile: `test/features/` — widget tests for all pages — POS: `test/` — checkout flow, sync engine • **Integration Tests**: — End-to-end order flow: browse → cart → checkout → payment → fulfillment — Family cart: create → invite → budget → collective checkout — POS: shift open → scan → pay → receipt → shift close • **Stress Tests**: — 1000 concurrent orders hitting inventory with optimistic locking — Verify no overselling (reserved_qty + sold ≤ quantity) — WebSocket load: 100 simultaneous POS connections • **Security Audit**: — RBAC: verify cashier can't access admin routes — API rate limiting: verify 429 responses under abuse — SQL injection testing on search endpoints — JWT token expiry & refresh flow — POS offline queue: verify orders can't be tampered with
BE	• Fix all bugs identified by QA • Optimize slow queries (add indexes, query plans) • Add API rate limiting middleware • Review & harden all RBAC guard configurations
FM	• Fix UI bugs and edge cases • Add error handling for all API failure scenarios • Implement retry logic for failed requests
FP	• Fix POS-specific bugs • Test with actual barcode scanner hardware • Validate receipt printing with thermal printer

Role	Tasks
AI	• Tune NLP model accuracy based on QA feedback • Optimize embedding search latency • Add fallback responses for unrecognized intents

Stage Gate: All critical/blocker bugs resolved. Unit test coverage ≥ 80%. No security vulnerabilities in OWASP Top 10. Stress tests pass without data corruption.

• Stage 8 — Deployment & Live Production

Duration: ~1 week · **Sprint 7**

Role	Tasks
PM	• Create launch checklist & go/no-go criteria • Coordinate store staff training schedule • Prepare rollback plan • Plan post-launch monitoring rotation
DO	• **Containerization**: — Finalize `Dockerfile.backend` (multi-stage build) — Build & push to container registry • **Infrastructure**: — Provision production PostgreSQL (managed: RDS/Cloud SQL) — Provision production Redis (ElastiCache/Memorystore) — Configure HTTPS, DNS, load balancer • **CI/CD**: — `cd-deploy.yml`: Automated prod deployment on tag push — Blue-green or rolling deployment strategy • **Monitoring**: — Deploy Prometheus + Grafana dashboards — Set up alerts: API latency > 500ms, error rate > 1%, DB connections — Configure log aggregation (ELK/Loki)
BE	• Run production migrations • Execute product seeder against production DB • Verify all environment variables are set • Smoke test all API endpoints on production domain
FM	• **App Store Release**: — Submit iOS build to TestFlight for Apple review — Submit Android build to Google Play Console — Prepare store listing: screenshots, descriptions (Arabic + English) • Final regression test on release builds
FP	• **POS Hardware Deployment**: — Compile Windows release binary — Flash/install on store POS machines — Pair Bluetooth barcode scanners — Configure thermal printers — Test end-to-end in store environment
AI	• Verify NLP model is deployed and responsive • Monitor AI inference latency on production hardware • Set up model performance logging
QA	• Production smoke tests across all platforms • Verify monitoring dashboards show healthy metrics • Sign off on launch readiness

Stage Gate: All systems live on production. Mobile apps submitted to app stores. POS running on store hardware. Monitoring dashboards active. Team on standby for 48h post-launch.

---

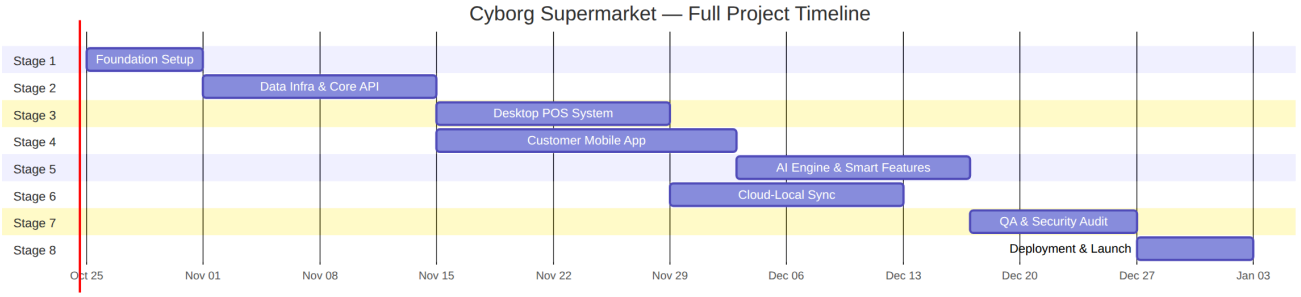
7. Data Model — Product Seed Mapping

The `super_market.xlsx` file contains **1000 products** ready to seed into the system:

Excel Column	Database Field	Type	Example
`barcode`	`product.barcode`	`BIGINT UNIQUE`	`6224000432065`
`ArabicPrtoductName`	`product.name_ar`	`VARCHAR(255)`	`مزارع دينا سمن نقي`
`EnglishProductName`	`product.name_en`	`VARCHAR(255)`	`Dina Farms Pure Ghee`
`ArabicCategory`	`category.name_ar`	`VARCHAR(100)`	`أطعمة طازجة`
`EnglishCategory`	`category.name_en`	`VARCHAR(100)`	`Fresh Food`
`ArabicSubCategory`	`category.name_ar` (child)	`VARCHAR(100)`	`منتجات الألبان والبيض`
`EnglishSubCategory`	`category.name_en` (child)	`VARCHAR(100)`	`Dairy & Eggs`
`brand`	`product.brand`	`VARCHAR(100)`	`Dina Farms`
`country`	`product.country`	`VARCHAR(50)`	`Egypt`
`size`	`product.size`	`VARCHAR(50)`	`600g`
`format`	`product.format`	`VARCHAR(50)`	`pieces`
`price`	`product.price`	`DECIMAL(10,2)`	`332.25`
`image`	`product.image_url`	`TEXT`	CDN URL
`embedding`	`product.embedding`	`VECTOR(1536)`	(to be generated in Stage 5)

Data Statistics: - **5 Categories**: Fresh Food, Beverages, Frozen Food, Food Cupboard, Fruits & Vegetables - **7 Subcategories**: Dairy & Eggs, Juices, Soft Drinks, Frozen Fruits & Vegetables, Condiments/Dressings/Marinades, Rice/Pasta/Pulses, Biscuits/Crackers/Cakes - **143 Brands** — all sourced from Egypt - **Prices**: In Egyptian Pounds (EGP)

8. Summary Timeline



> **Stages 3 and 4 can run in parallel** after Stage 2, since the POS (Stage 3) and Mobile App (Stage 4) are independent client applications consuming the same backend APIs. Stage 6 (Sync) depends on both being complete.

Estimated Total Duration: ~11–13 weeks (with parallel execution of Stages 3 & 4)